

Technical Summary: Real-Time WebSocket Engine (Phase 1)

This document serves as a comprehensive technical breakdown of the chat engine architecture built so far, moving from core connectivity to production-grade system engineering.

Part 1: Core Architecture Implemented

1. WebSocket Server Initialization

The foundation of the system is a hybrid **Express + Socket.io** server. Unlike standard REST APIs that use a Request-Response cycle, this establishes a **Persistent Full-Duplex Connection**.

- **Internal Flow:** Browser → HTTP Handshake → Protocol Upgrade (WS) → Persistent Socket.
- **Key Identifier:** Each connection is assigned a unique `socket.id` (e.g., `kf83hdf93`), allowing the server to track individual sessions.
- **Real-World Utility:** This is the baseline for platforms like Discord, Slack, and high-frequency trading apps.

2. Event-Driven Communication

We moved away from static endpoints to an **Asynchronous Event Model**.

- **Mechanism:** Using `.emit()` to send data and `.on()` to listen.
- **Structure:** Centralized in `socket.events.js` for maintainability.
- **Benefit:** Enables non-blocking, high-speed data transfer.

3. Room-Based Logic & Broadcasting

The server manages "Rooms"—virtual groupings of socket IDs.

- **Logic:** `socket.join(roomId)` maps a user to a specific topic.
- **Broadcasting:** `io.to(roomId).emit(...)` ensures messages are only sent to relevant participants, optimizing bandwidth and privacy.
- **Analogy:** This functions exactly like a Slack Channel or a WhatsApp Group.

4. Modular Handler Design

Logic is decoupled into specific directories:

- `handlers/`: Business logic for messages and connections.
- `events/`: Standardized event name definitions.

- socket.js: Server configuration.
- **Why:** Prevents "Spaghetti Code" and allows the system to scale to thousands of lines without becoming unmanageable.

Part 2: Production-Grade Enhancements

Current Focus: Moving from a "Project" to a "Product."

Feature	Technical Implementation	Purpose
Health Checks	GET /health endpoint	Allows Kubernetes/Load Balancers to monitor uptime.
User Tracking	socketId ↔ username mapping	Enables the server to know <i>who</i> is behind a connection.
Presence System	disconnect event listeners	Tracks online/offline status in real-time.
Winston Logging	Level-based logging (info, error)	Replaces console.log for persistent, searchable error tracking.
Input Validation	Schema checks for roomId & message	Prevents server crashes from malformed or malicious data.

The Next Evolution: System Engineering

We are transitioning into **Phase 1, Part 2**, where the system gains "State Awareness."

Upcoming: The User Registry Service

We will implement `services/userRegistry.js` to manage the complex relationships between users, sockets, and rooms.

Key Objectives for the Next Phase:

1. **Observability:** Knowing exactly what happens via structured logs.
2. **State Management:** Maintaining an active list of users per room.
3. **Stability:** Ensuring the server handles unexpected disconnections gracefully.

Note for Docs: This architecture is designed to be **Database Agnostic**. Currently, all state is in-memory for maximum speed, preparing the foundation for future integration with Redis or PostgreSQL.